Level 2 Viva Script

Smart MCQ Solver Challenge — DL & GenAI Project [BSDA2001P]

Aryan (24f1001839) | Duration: 60 minutes | Format: Industry-panel technical viva

> **HOW TO USE THIS DOCUMENT.** This is not a script to memorize word-for-word and recite — a Level 2 industry panel will ask unscripted follow-up questions the moment they sense recitation, and rule 4 requires you to actually write code live. Every section has two kinds of text:
>
> > •**Yellow "You:" boxes** — a natural spoken narration you can say close to as-written once you understand what it means.
> >
> > •**Blue "STUDY NOTE" boxes** — the underlying concept explained in plain language, with zero assumed prior knowledge. Read these *first*, until the ideas feel like your own, before touching the yellow boxes. If you can re-explain a blue box in your own words to a friend, you own that section.
>
> Read this whole document twice. First pass: understand every blue box. Second pass: read the yellow boxes aloud, timing yourself. Then go straight to **Appendix A** and actually type the two functions there from memory ten times each — that is the single highest-leverage thing you can do in the next hour, because it is the one part of the viva no document can do for you in the room.

60-Minute Timing Plan

1. Opening & Problem Framing — **0:00–0:05**
2. The Journey — 15 Failed Attempts and What They Taught Me — **0:05–0:18**
3. Exploratory Data Analysis Deep Dive — **0:18–0:28**

4. Code Walkthrough — Setup → Model 1 → Model 2 → Model 3 — **0:28–0:48**
5. Weights & Biases Tracking & Metric Comparison — **0:48–0:52**
6. Ensemble, Error Analysis & Final Result — **0:52–0:58**
7. Closing Statement — **0:58–1:00**

Appendices (study before the viva, not spoken as a block): A — Live-coding practice · B — Anticipated Level-2 Q&A · C — Git/GitHub crash course · D — Weights & Biases crash course · E — Glossary

Part 1 — Opening & Problem Framing (0:00–0:05)

> ***You:*** *"Good morning/afternoon. I'll be walking you through my project for the Smart MCQ Solver Challenge — a private Kaggle competition where the task is: given a question prompt and five answer options labelled A through E, predict the top three most likely correct answers, in ranked order. It's scored with* ***Mean Average Precision at 3****, or MAP@3 — a metric that rewards you more for getting the correct answer at rank 1 than at rank 3, and gives zero credit if the correct answer isn't in your top three at all.*

> *The brief required at least three distinct models — one built entirely from scratch, one pretrained model from the BERT/RoBERTa family, and one additional model of my choice — all tracked in Weights & Biases, compared on common metrics, with a minimum public leaderboard score of 0.73 to qualify.*

> *Before I show you my final notebook, I want to be upfront: this wasn't a straight line. I*

> *went through roughly fifteen documented experiment iterations before I found the formulation that actually worked, and I think walking you through why those failed is more informative than just showing you the version that succeeded — it's the part where the actual learning happened."*

> **STUDY NOTE — MAP@3 in one sentence.** For each question, look at your ranked top-3 guesses. If the correct label is your 1st guess, you get 1 point. If it's your 2nd guess, you get 1/2 a point. If it's your 3rd guess, you get 1/3 of a point. If it's not in your top 3, you get 0. Average that score across every question in the test set. That's MAP@3. It punishes confident wrong answers less than it punishes not even considering the right answer.

Part 2 — The Journey: 15 Failed Attempts (0:05–0:18)

> ***You:*** *"My repository has a `trials/` folder with the full execution logs of every earlier attempt — I kept them deliberately instead of deleting them, because they document the actual debugging journey. Every single one of those fifteen runs used the same fundamentally wrong formulation: I treated this as a standard 5-class text classification problem. I concatenated the prompt and all five options into one long string — something like `prompt [SEP] A: ... B: ... C: ... D: ... E: ...` — and fed that whole block into `AutoModelForSequenceClassification` with 5 output labels.*

> *Every one of those fifteen runs scored between 0.687 and 0.72 on the public leaderboard. Every one is marked FAILED in its own log, because the pass bar was 0.73. I tried swapping in RoBERTa-base, RoBERTa-large, DeBERTa-v3-small, DeBERTa-v3-base, ALBERT-base-v2, in every combination; I tried LoRA fine-tuning with the `peft` library; I tried prompt-cleaning with regex to strip boilerplate; I tried de-duplicating my validation set. None of it broke through 0.72, because none of it fixed the actual problem — the formulation itself."*

2.1 — The two root causes

> **STUDY NOTE — Root cause #1: truncation.** A tokenizer converts text into numbers (tokens) a model can read, and every model has a hard cap on how many tokens it will accept — I was using `max_length=128`. When you glue a prompt plus five full-sentence options together, that block runs 250 to 650 tokens long on average. Anything past token 128 gets silently cut off.

> In practice, that meant options D and E — the last two — were very often *never seen by the model at all*. I wasn't training a model that struggled to compare five options; I was training a model that could only see three of them.

> **STUDY NOTE — Root cause #2: wrong inductive bias.** "Inductive bias" just means: what structural assumption does the architecture bake in? A sequence classifier reads one big block of text and outputs one probability distribution over 5 fixed classes — it never explicitly compares option A against option B; it just learns "this whole blob of text → class 2" as one flat pattern. That's the right tool when the five classes mean something fixed and stable across every example (like "sentiment: positive/negative/neutral"). But here, "option A" isn't a fixed concept — row 1's option A is about Heidegger, row 2's option A is about nuclear fusion. The model has no way to structurally represent "compare these two competing pieces of text." It was memorizing surface patterns in training data, not reasoning about the actual question.

> ***You:*** *"So even my best-tuned sequence-classification run — with LoRA, cleaned prompts, and a de-duplicated validation set — hit a validation MAP@3 of 0.84, which looked fantastic, but scored only 0.687 on the actual leaderboard. That gap is the single most important data point in my whole project, and I want to explain exactly why it happened, because it's a textbook* ***data leakage*** *story."*

2.2 — The leakage story (the most important debugging insight)

> **STUDY NOTE — Why 0.84 validation became 0.687 leaderboard.** The training set has 2,000 rows but only about 250 genuinely distinct *questions* — each real question is repeated roughly 8 times, wrapped in a different boilerplate prefix/suffix each time (e.g. "Pick the best possible answer: ..." vs "Which of the following is correct? ..."), and sometimes with tiny synonym swaps in the distractor options ("light-ion" vs "heavy-ion"). If you split randomly into train/validation, copies of the *same underlying question* land on both sides. The model doesn't need to reason about physics — it just needs to recognize "I've seen this exact question's phrasing before" and recall the answer it memorized. That's why validation looked amazing: the model was echoing memorized answers, not generalizing. On the leaderboard's genuinely unseen questions, that memorization was worthless, so the score collapsed back down to the same ~0.70 every sequence-classification attempt landed at. I even had one run where I tried to fix this by grouping the split — but I grouped on the raw *option text* instead of the *cleaned question text*, so the leak survived anyway. That specific bug is why v96 and v98 in my trials folder show a validation score of 0.84 sitting right next to a leaderboard score of 0.687 — I keep that pair of numbers as the clearest evidence in my whole project of why validation

methodology matters as much as model choice."

> ***You:*** *"That failure is what forced me to stop tuning hyperparameters and go back to the data itself. I ran a proper EDA, and that EDA is what changed my entire approach — which is what I'll walk you through next."*

Part 3 — Exploratory Data Analysis Deep Dive (0:18–0:28)

> ***You:*** *"My final notebook's EDA section asks five questions of the data, and each one directly produced a design decision. Let me go through them."*

3.1 — Completeness & class balance

> ***You:*** *"First, missing values — there are none, in either train or test. Second, the label distribution: I computed how often A, B, C, D, E are each the correct answer, and it ranges from about 16% to 25% — mildly imbalanced, nowhere near severe. That told me I didn't need SMOTE, oversampling, or class-weighted loss — those would have added complexity for a problem that didn't exist. Simple cross-entropy was the right call."*

3.2 — Text length → choosing `max_length`

> ***You:*** *"This is the cell that directly diagnoses the truncation bug from my earlier attempts. I measured prompt length, single-option length, prompt+one-option length, and prompt+all-five-options length, in characters, and converted roughly to tokens. Packing all five options together needs on average ~230 tokens and up to ~650 in the worst case — well past my old `max_length=128`. But scoring just one option against the question at a time needs only about 130 tokens at the 95th percentile. That's the number that told me `max_length=256` would comfortably fit everything if I restructured the problem to score one option at a time instead of all five at once."*

3.3 — Hidden structure in the prompt (the boilerplate)

> ***You:*** *"I wrote a small regex to pull out the first sentence-like chunk before a colon or question mark, and the last few words after the final question mark, across the whole dataset. What came back was exactly six recurring prefixes — things like 'Pick the best possible answer:', 'Determine the correct option:', 'Which of the following is correct?' —*

> *crossed with exactly four recurring suffixes like 'among the listed options.' or 'carefully.'. That's 24 surface variants of instruction-text carrying zero actual information about the question. I stripped it identically from both train and test."*

3.4 — The key finding: only ~252 unique questions

> ***You:*** *"This was my biggest single insight. After stripping the boilerplate, I counted unique cleaned prompts: out of 2,000 training rows, there are only about 252 genuinely distinct questions — each one repeated on average 7 to 8 times behind a different wrapper. I checked whether the copies ever disagreed on the correct answer — they never did, the labels are perfectly clean — so keeping every copy is free data augmentation, not noise, as long as my train/validation split respects that grouping. That's exactly the fix for the leakage problem I hit earlier: I now group my split by the cleaned question text, so every copy of a question — regardless of which wrapper it's dressed in — stays entirely on one side of the split. I assert that with code, not just intent — there's a line that explicitly checks the intersection between train and validation question sets is empty, and raises an error if it isn't."*

3.5 — The second noise type: synonym-swapped distractors

> ***You:*** *"I picked a repeated question where I could see the option text itself varying between copies, and used Python's `difflib` to diff the strings character-by-character. What that surfaced is a second, more subtle kind of variation: distractor options get small synonym or number swaps between copies — 'light-ion' vs 'heavy-ion', '10 kV' vs '100 kV', 'method' vs 'approach'. The meaning is preserved in the correct answer, but a distractor sometimes becomes a slightly different flavor of wrong. This confirmed to me that the model has to read every option in full — no truncation is survivable here, because the difference between right and wrong is sometimes two or three words."*

3.6 — Why TF-IDF fails, and the length-only baseline

> ***You:*** *"Before touching any transformer, I built two baselines to know what I actually had to beat. I computed TF-IDF cosine similarity between the prompt and each option, expecting some weak signal — instead it scored below random guessing, around 0.33. That makes sense in hindsight: all five options for a given question share almost the exact same vocabulary, since the wrong answers are near-paraphrases of the right one. Bag-of-*

> *words literally cannot tell them apart; it needs a model that understands meaning, not just word overlap — hence a deep cross-encoder was mandatory, not optional. My second baseline — always guessing the longest option — actually reached about 0.58, a real but shallow signal I later reused as a tie-breaker in my final ensemble for low-confidence tail predictions."*

> ***You:*** *"Putting all of that together into one design decision: instead of one 5-way classifier that reads a 650-token blob and never really compares anything, I reformulate the task so each option is scored separately against the question by one shared encoder, and a softmax over the five resulting scores gives the ranking. That is precisely what Hugging Face's* `AutoModelForMultipleChoice` *head is built for."*

Part 4 — Code Walkthrough (0:28–0:48)

4.1 — Setup & reproducibility

> ***You:*** *"My setup cell fixes a random seed — 42 — across Python's `random`, NumPy, and PyTorch, including the CUDA seed, so re-running the notebook gives comparable results run to run. I detect whether a GPU is available and print the device name; on Kaggle this is a T4."*

> **STUDY NOTE.** A "seed" just initializes the random-number generator to a fixed starting point, so operations that look random (shuffling data, initializing weights) are actually reproducible — same seed, same sequence of "random" numbers, every run.

4.2 — Weights & Biases login

> ***You:*** *"Next I set up Weights and Biases logging. On Kaggle I read a secret called `WANDB_API_KEY` through `UserSecretsClient`; if that's missing, I fall back to offline mode instead of letting the whole run crash — logging is important, but it should never be the thing that breaks an otherwise-working training run."*

4.3 — Preprocessing & the group-aware split

> ***You:*** *"I apply the same `clean_prompt` function from EDA to both train and test — stripping the six prefixes and four suffixes, run twice per string because a small number of rows had the wrapper applied twice. Then the split: I take the unique cleaned questions, shuffle them with the fixed seed, and hold out 20% of unique questions — not rows — for validation. Every row belonging to a held-out question goes to validation; everything else goes to train. I assert in code that the intersection of train and validation question sets is empty. That assertion is the single most important line of defense against the exact leakage bug that sank my earlier attempts."*

4.4 — The `encode()` function (this is the one to know cold)

```
def encode(df, tok):
```

```
    # -> (n, 5, MAX_LEN): every option paired with its question


    first  = [q for q in df.question for _ in range(5)]


    second = [str(r[c]) for _, r in df.iterrows() for c in LETTERS]


    e = tok(first, second, truncation=True, max_length=MAX_LEN,


            padding="max_length", return_tensors="pt")


    n = len(df)


    return e["input_ids"].view(n, 5, MAX_LEN), e["attention_mask"].view(n, 5,
MAX_LEN)
```

> ***You:*** *"This is the heart of the whole reformulation, so let me be precise about it. For every row, I duplicate the cleaned question string five times, and pair it — in order — with options A, B, C, D, E. The tokenizer receives two parallel lists and encodes them as five separate `(question, option)` pairs, exactly the same way you'd tokenize two-sentence input for something like natural language inference. I get back a flat batch of `n×5` encoded sequences, and I `.view()` that back into shape `(n, 5, MAX_LEN)` — `n` questions, 5 choices each, MAX_LEN tokens per choice. That's exactly the input shape `AutoModelForMultipleChoice` expects: it internally flattens the 5-choice dimension back into the batch dimension, runs all of them through one shared encoder, then un-flattens to produce 5 logits per question and softmaxes across them."*

4.5 — Model 1: the from-scratch Transformer

```
class ScratchMCQ(nn.Module):


    def __init__(self, vocab, pad_id, d=256, heads=4, layers=4, ff=512,
drop=0.1):


        super().__init__()
```

```
        self.tok = nn.Embedding(vocab, d, padding_idx=pad_id)

        self.pos = nn.Embedding(MAX_LEN, d)

        self.enc = nn.TransformerEncoder(

            nn.TransformerEncoderLayer(d, heads, ff, dropout=drop,

                                       batch_first=True, norm_first=True),
layers)

        self.norm, self.drop = nn.LayerNorm(d), nn.Dropout(drop)

        self.score = nn.Linear(d, 1)




    def forward(self, ids, mask):

        B, C, L = ids.shape

        ids, mask = ids.view(B * C, L), mask.view(B * C, L)

        h = self.tok(ids) + self.pos(torch.arange(L, device=ids.device))

        h = self.enc(h, src_key_padding_mask=(mask == 0))

        h = (h * mask.unsqueeze(-1)).sum(1) / mask.sum(1,
keepdim=True).clamp(min=1)

        return self.score(self.drop(self.norm(h))).view(B, C)
```

> ***You:*** *"This is a 4-layer, 256-dimensional Transformer encoder with 4 attention heads, initialized with random weights — no pretraining at all. It has to use the exact same architectural pattern as my pretrained models — one shared encoder scores each*

> *(question, option) pair, softmax over the 5 scores — so that the comparison between models 1 and 2/3 isolates one single variable: does pretrained language knowledge matter here?*

> *Walking through the forward pass: I flatten the batch-of-5-choices dimension into the batch dimension so I can run everything through the encoder as one flat batch of sequences. I add a learned token embedding to a learned positional embedding — the model has no idea about word order otherwise, since attention itself is permutation-invariant. I run it through the Transformer encoder stack, using the attention mask to tell the model which tokens are real versus padding. Then — this is the key step — I mean-pool over the sequence: I multiply each token's hidden state by its mask (zeroing out padding), sum, and divide by the count of real tokens, giving one fixed-size vector per (question, option) pair regardless of how long that sequence actually was. A final linear layer maps that vector to a single real number — a 'score' for that option. I un-flatten back to shape `(B, 5)`, giving five raw scores per question, which get softmaxed into probabilities.*

> *Result: this model badly underperforms — around a MAP@3 of 0.57, barely above the length-only baseline. And that's actually the correct, expected, informative result: with only ~252 unique training questions, there is nowhere near enough data for a randomly-initialized model to learn the physics and philosophy semantics this dataset requires from scratch. It empirically proves that pretrained knowledge isn't a nice-to-have here — it's mandatory."*

4.6 — Model 2: RoBERTa-base (the pretrained BERT-family model)

> ***You:*** *"Model 2 is `roberta-base`, loaded through `AutoModelForMultipleChoice.from_pretrained` — same architecture pattern as before, but now the encoder starts from weights pretrained on about 160GB of text instead of random initialization. I explicitly cast it to `.float()` — full 32-bit precision — before training, and I want to explain why, because it's a real bug I hit and fixed. Newer versions of the `transformers` library load a checkpoint in whatever dtype it was saved in. Some checkpoints — DeBERTa-v3-base among them — are saved in fp16. If your master weights are fp16 but you're using PyTorch's automatic mixed-precision `GradScaler` for*

stability, you get a runtime crash: 'Attempting to unscale FP16 gradients.' Forcing `.float()` right after loading keeps the master weights in fp32 so autocast can safely manage the mixed-precision math around them.

I train for 8 epochs with a learning rate of 2e-5, batch size 4 with gradient accumulation of 2 for an effective batch of 8 — small because DeBERTa/RoBERTa at this sequence length and 5-way expansion takes real VRAM. Result documented in my methodology: roughly 0.83 MAP@3 on my honest, leak-free validation set — a huge jump over the scratch model, and proof that pretrained representations transfer well to this domain even with full fine-tuning on just ~250 underlying questions."

4.7 — Model 3: DeBERTa-v3-base (my model of choice)

You: *"For my third, freely-chosen model I picked `microsoft/deberta-v3-base`, and I chose it for a specific technical reason, not arbitrarily. DeBERTa-v3 replaces BERT/RoBERTa's absolute position embeddings with disentangled attention — it keeps content and position information in separate vectors and combines them inside the attention score, rather than adding a position vector directly onto the content vector. And critically, it's pretrained with an* ***ELECTRA-style replaced-token-detection*** *objective instead of masked-language-modeling: during pretraining, it explicitly learns to detect which tokens in a sentence have been swapped for a plausible-but-wrong alternative. That is structurally the same skill this competition is testing — our distractors are, almost literally, replaced tokens: '10 kV' swapped for '100 kV', 'light-ion' swapped for 'heavy-ion'. I'm not just picking a bigger model; I'm picking a model whose pretraining objective is mathematically aligned with the exact failure mode of the wrong answers in this dataset. It was consistently my strongest individual model."*

4.8 — The shared training loop & a subtle bug I had to fix

You: *"All three models share one `train_model` function so the comparison is fair — same optimizer (AdamW with weight decay 0.01), same linear warmup schedule over 10% of steps, same gradient clipping at norm 1.0, same mixed-precision scaler. One detail I want to call out because it's exactly the kind of bug a live-coding question might probe: my `forward_logits` helper calls the pretrained models with keyword arguments only*

> *— `model(input_ids=ids, attention_mask=mask)` — never positionally. That's not stylistic preference. RoBERTa's forward signature takes `token_type_ids` as its second positional argument. If I'd called `model(ids, mask)`, the attention mask would silently land in the `token_type_ids` slot instead — and because RoBERTa's `type_vocab_size` is only 1, any value except 0 there causes an index-out-of-bounds crash during embedding lookup. It's a real trap in the Hugging Face API, and I found it the hard way."*

Part 5 — Weights & Biases Tracking (0:48–0:52)

> ***You:*** *"Every one of my three training runs calls `wandb.init()` with a project name and a config dictionary — model name, learning rate, batch size, epochs — so each run is a fully reproducible, timestamped record, not just a number I remember. Inside every epoch, I log four numbers: training loss, validation MAP@3, validation macro-F1, and the epoch index. That gave me live curves I could actually watch — for example, it's how I visually confirmed the scratch model's training loss kept dropping toward zero while its validation MAP@3 plateaued and even wobbled downward after epoch 3 or 4 — the textbook shape of overfitting — while DeBERTa's validation curve kept climbing smoothly alongside its training loss, meaning it was actually generalizing.*

> *At the end, I build a single pandas comparison table across all three runs — MAP@3, accuracy, macro-F1 side by side — and log that table into a fourth, dedicated 'comparison' W&B run, so the three-way comparison the brief requires lives as one clean artifact in the project, not scattered across separate dashboards."*

> **STUDY NOTE — What Weights & Biases actually is, for the Q&A.** It's an experiment-tracking dashboard. Instead of scrolling back through printed console logs to remember "which run had a learning rate of 2e-5 and got 0.83", every call to `wandb.log()` during training streams that number to a hosted dashboard, tagged with the run's config. You get a live line chart per metric per run, and you can overlay multiple runs to compare them visually. It solves the very real problem of experiment amnesia — running 15 variants and losing track of what changed between them, which is exactly the trap most of my `trials/` folder fell into before I disciplined myself to log everything.

Part 6 — Ensemble, Error Analysis & Final Result (0:52–0:58)

6.1 — Error analysis on my best single model

> ***You:*** *"Before ensembling, I ran error analysis on DeBERTa-v3-base, my strongest single model. I computed per-true-label accuracy — how well the model does specifically on questions where A is correct, versus B, versus C, and so on — and a full confusion*

> *matrix of true label vs predicted label. I also broke MAP@3 down by rank: what fraction of the time was the correct answer my #1 guess, my #2 guess, my #3 guess, or missed entirely — which tells you not just how often you're right, but how confidently. Finally I pulled two specific wrong predictions and printed the true option next to the predicted option side by side. In every case I inspected, the two options the model confused were separated by exactly one qualifying word or number — the kind of distinction that genuinely requires outside factual knowledge to break, not more fine-tuning on the same 250 questions. That's an honest limitation I can defend if asked, not a hidden one."*

6.2 — The ensemble

> ***You:*** *"The two pretrained models make different mistakes on different questions, so averaging their predicted probabilities should help — and it does, but I don't average blindly. I run a grid search over the weight given to each of the three models, evaluated on my grouped validation set, and let the search itself decide the weights rather than guessing them. In every version of this search, the scratch model earns a weight close to zero — it's not being penalized arbitrarily, the search is telling me, honestly, that it doesn't generalize and shouldn't be trusted in the blend, even though it's a fully-trained, fully-tracked model satisfying the 'from scratch' requirement on its own."*

6.3 — Retraining on 100% of the data & the final submission

> ***You:*** *"The 20% of questions I held out earlier were genuinely useful training signal, so for my actual Kaggle submission I retrain RoBERTa-base and DeBERTa-v3-base on the full 2,000 rows using the hyperparameters I'd just validated, and I add a fourth, larger model — `microsoft/deberta-v3-large` — trained with gradient checkpointing and a small per-device batch size to fit T4 memory, since it's meaningfully bigger and has a real chance at closing the remaining gap.*

> *For the final blend, rank 1 of my prediction comes from a weighted average of the full-data RoBERTa, full-data DeBERTa-base, and the large model — the large model gets the heaviest weight, 0.5, since it's the strongest. For ranks 2 and 3 — the 'tail' — I blend the large model's probabilities with my normalized-option-length prior from the EDA baseline, on the reasoning that when the model itself is uncertain about second and third*

> *choices, that shallow-but-real length signal adds a small amount of independent information rather than pure noise.*

> *That output — three distinct predicted labels per question, submitted as `ID,Prediction` exactly matching the sample submission format — is what crosses the 0.73 cutoff on the public leaderboard."*

Part 7 — Closing Statement (0:58–1:00)

> ***You:*** *"If I had to compress this whole project into one lesson: throwing a bigger model at a hard problem is not a strategy — understanding your data is. Every fix that actually moved my score came directly out of EDA: fixing sequence truncation came from measuring token lengths; fixing the false 0.85-validation-vs-0.69-leaderboard gap came from realizing my split wasn't grouped correctly; choosing DeBERTa over a bigger but less-suited model came from understanding its pretraining objective, not its parameter count. The architecture change from sequence-classification to multiple-choice scoring — and the honest, leak-free evaluation that let me actually trust my own validation numbers — is what took me from fifteen failed attempts stuck at 0.70 to a result that clears the bar. I'm happy to open any cell and go deeper, or write out a function live if you'd like."*

Appendix A — Live-Coding Practice *(do this now, by hand, on paper or in a blank file)*

> **This is the highest-priority section in the whole document.** Rule 4 requires you to write something live. Type each of these from memory, from scratch, no copy-paste, at least 10 times each until your hands know it without looking back at this page. If asked to write something else entirely, these two mental models — "loop through predictions, check rank, add 1/(rank+1)" and "argsort descending, take top-3" — cover almost anything they'd plausibly ask for at this level.

A.1 — MAP@3 from scratch (the metric itself)

```
def map_at_3(predictions, true_labels):

    score = 0.0

    for pred_top3, true_label in zip(predictions, true_labels):

        for rank, pred_label in enumerate(pred_top3):

            if pred_label == true_label:

                score += 1.0 / (rank + 1)

                break

    return score / len(true_labels)
```

> **Say this while you type it:** "For every question, I walk through my three ranked guesses. The moment I find the true label, I add 1 divided by (its rank plus 1) — rank 0 gives 1.0, rank 1 gives 0.5, rank 2 gives 0.333 — and I stop looking, because I only credit the *first* correct hit. If I never find it in the loop, I add nothing. Then I average over every question."

A.2 — Turning raw model scores into a ranked top-3 string

```
import numpy as np


LETTERS = list("ABCDE")




def top3(probs_row):


    # probs_row: 5 numbers, one per option A..E


    order = np.argsort(-probs_row)[:3]        # indices of 3 largest,
descending


    return " ".join(LETTERS[i] for i in order)
```

> **Say this while you type it:** "`argsort` gives me the indices that would sort the array ascending; negating the array first flips that to descending, so index 0 is my highest-probability option. I slice the first three, and map each index back to its letter."

A.3 — If asked to write the mean-pooling step by hand (in words, not necessarily code)

> "I multiply each token's hidden vector by its attention-mask value — 1 for a real token, 0 for padding — so padding contributes nothing. I sum across the sequence dimension, then divide by the count of real tokens (the sum of the mask), so a 10-token option and a 100-token option both end up as one comparable fixed-size vector, not skewed by length."

Appendix B — Anticipated Level-2 Q&A

Q: Why not use LoRA in the final notebook, if you tried it in trials?

> *"In earlier trials — v95 and v97 in my logs — I did use PEFT/LoRA on DeBERTa and RoBERTa. But at this dataset size, ~2,000 rows on base-sized models, full fine-tuning only took about 7 minutes per model on a T4 — there was no memory or time pressure LoRA needed to relieve. And empirically, in my own trial logs, full fine-tuning adapted the encoder more aggressively to this narrow scientific-question domain than the ~0.3%-of-parameters LoRA update did, so I favored it for the version that had to actually clear the leaderboard bar."*

Q: How did you handle class imbalance?

> *"I plotted the true-answer distribution across A–E during EDA and it ranged 16% to 25% — mild imbalance. I judged that SMOTE or class-weighted loss would add complexity and risk without addressing a real problem, so I used plain cross-entropy."*

Q: Did you consider RAG or external knowledge?

> *"I deliberately didn't use it in the final submission. My error analysis showed the pretrained models already carry enough general knowledge to solve these questions once the input format lets them actually see and compare every option — the bottleneck was formulation, not missing knowledge. Adding a retrieval pipeline would have added real complexity for a problem EDA showed wasn't the actual constraint."*

Q: Explain the difference between AutoModelForSequenceClassification and AutoModelForMultipleChoice precisely.

> *"SequenceClassification takes one input sequence and outputs a fixed-size probability distribution over N pre-defined classes — it's the right tool when 'class 2' means the same thing across every example, like a sentiment label. MultipleChoice takes N separate (context, choice) pairs sharing one encoder, produces one scalar score per choice, and softmaxes across just those N scores for that specific example — the 'classes' are example-specific text, not fixed categories. It's structurally built for exactly this kind of question."*

Q: What is gradient checkpointing and why did you need it for the large model?

> *"Normally, backpropagation needs to keep every layer's intermediate activation in memory to compute gradients. Gradient checkpointing throws most of those away during the forward pass and recomputes them on demand during the backward pass — trading extra compute time for much lower peak memory. I needed it for deberta-v3-large specifically because it's large enough that keeping all activations for a batch would exceed T4 VRAM at a usable batch size."*

Q: Why fp16/mixed precision, and what's the master-weights bug you mentioned?

> *"Mixed precision runs most operations in 16-bit floats for speed and memory savings, while keeping a 32-bit 'master copy' of the weights for numerically stable updates — PyTorch's GradScaler manages that automatically. The bug: newer transformers versions load a checkpoint in whatever precision it was saved in, and some checkpoints ship in fp16. If your master weights are already fp16 when GradScaler tries to unscale fp16 gradients, it raises an explicit error, because the whole scheme assumes fp32 master weights. Calling `.float()` right after loading fixes it."*

Q: What's the single biggest mistake in your journey, and what does it tell you about ML methodology generally?

> *"Trusting a validation score without first verifying my split had no leakage. It taught me that a validation number is only as trustworthy as the process that produced the split — an impressive metric with a broken evaluation setup is worse than a modest metric with an honest one, because the impressive one actively misleads every decision you make afterward."*

Q: Walk me through what happens if the attention_mask were accidentally all zeros for a row.

> *"In my mean-pooling step, the denominator — sum of the mask — would be zero, which is a division by zero. That's exactly why I clamp it with `.clamp(min=1)` in the scratch model's forward pass — it's a defensive guard against an empty-sequence edge case, even though in practice every real row has at least some non-padding tokens."*

Appendix C — Git & GitHub, Explained From Zero

> **What a "repository" is:** a folder whose entire history of changes is tracked, not just its current contents. Every saved checkpoint of that history is called a **commit** — a snapshot, with a message describing what changed and when.

> **What "git log" shows:** the list of commits, newest first, each with a short ID, an author, a date, and a message. If asked to show progress, this is literally the timeline of your work.

> **How my own progress is recorded:** I wrote a helper script, `kaggle.sh`, with commands like `pull`, `push`, `status`, `submit`. The `push` command uploads my notebook to Kaggle to run on their GPU, then automatically stages the updated notebook file and its kernel metadata and commits them with a timestamped message like "Update Kaggle notebook and kernel environment (date time)". That's why my git history shows a real cadence of dated commits — each one corresponds to an actual iteration I pushed and evaluated on the leaderboard, not a single last-minute upload.

> **If asked "what does `git diff` do" or similar:** it shows the line-by-line difference between two commits, or between your current uncommitted changes and the last commit — the same "before vs after" view you'd want when explaining what changed between two experiment versions.

> **Be honest about this if pressed:** if your examiner checks the actual commit timeline and it's shorter than three weeks, don't argue the point — say plainly that your GPU-bound iteration happened in a concentrated window, and pivot immediately to demonstrating depth of understanding of what changed between iterations, which is what rule 3 is really trying to verify.

Appendix D — Weights & Biases, Explained From Zero

> **What it is:** a hosted dashboard for machine learning experiments. Think of it as a logbook that draws live charts for you instead of you scrolling through console text.

> **The three calls that matter:**

- `wandb.init(project=..., name=..., config={...})` — starts a new tracked "run"

> and records the hyperparameters you used for it.

- `wandb.log({"train_loss": x, "val_map3": y, ...})` — called once per epoch (or per step); each call adds one point to that metric's line chart for the current run.
- `wandb.finish()` — closes out the run and syncs everything to the dashboard.

> **Why it matters for this project specifically:** the brief requires comparing at least three runs on common metrics. Because every model logs the same metric names (`val_map3`, `val_macro_f1`), I can overlay all three runs on one chart in the dashboard and visually compare convergence speed and overfitting behavior, not just a single final number.

Appendix E — Glossary (for anything else that comes up)

Term	Plain-language meaning
Tokenizer	Converts text into a sequence of integer IDs a model can process; also handles truncation/padding to a fixed length.
Attention mask	A same-length array of 1s and 0s marking which tokens are real content (1) versus padding (0), so the model ignores padding.
Softmax	Converts a vector of raw scores into probabilities that sum to 1, emphasizing the largest scores.
Cross-entropy loss	The standard loss for classification: penalizes the model more the further its predicted probability for the correct class is from 1.
Learning rate	How large a step the optimizer takes when updating weights each batch; too high diverges, too low trains too slowly.
Warmup schedule	Start the learning rate near zero and ramp it up over the first ~10% of training steps, to avoid destabilizing

	randomly-initialized layers early on.
Gradient clipping	Caps the size of the gradient update to prevent occasional huge, destabilizing steps.
Epoch	One full pass through the entire training dataset.
Macro-F1	The F1 score (harmonic mean of precision and recall) computed separately per class, then averaged unweighted — so a rare class counts as much as a common one.
LoRA	Low-Rank Adaptation: instead of updating all of a pretrained model's weights, freeze them and train two small "low-rank" matrices per target layer, drastically cutting trainable parameters.
Disentangled attention (DeBERTa)	Keeps a token's content vector and position vector separate, combining them only inside the attention score, rather than adding them together up front like BERT/RoBERTa.
ELECTRA-style pretraining	Pretrain by detecting which tokens in a sentence were swapped for plausible-but-wrong replacements, rather than only predicting masked-out tokens.